Національний технічний університет України

«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформаційних систем та технологій

Лабораторна робота № 4

з дисципліни «Протоколи та алгоритми електронного голосування»

Виконав: студент групи ІП-34 Туболев М. Р.

Перевірив: Асистент кафедри, PhD Нестерук А. О.

2026

1. Тема, мета та завдання роботи

Тема: розподіл повноважень у протоколах Е-голосування. Частина 2.

Мета роботи: програмно реалізувати протокол електронного голосування з розділенням бюлетеня між двома незалежними виборчими комісіями (ВК-1 та ВК-2) і центральною виборчою комісією (ЦВК), у якому беруть участь два кандидати та п’ять виборців; дослідити використання мультиплікативної гомоморфності RSA для об’єднання зашифрованих частин бюлетеня та можливості виявлення шахрайства окремої комісії.

Завдання роботи:

- реалізувати функціонал ЦВК, ВК-1 та ВК-2 у вигляді окремих програмних модулів та обмін даними між ними відповідно до протоколу;
- реалізувати самостійну генерацію кожним виборцем пари ключів RSA для ЕЦП і передачу відкритого ключа до ЦВК;
- реалізувати формування ЦВК списку виборців і кандидатів, надання кандидатам ID, придатних для поділу на множники, і публікацію цих ID;
- реалізувати генерацію ЦВК пари ключів RSA для шифрування бюлетенів, публікацію відкритого ключа та передачу списку виборців до ВК-1 і ВК-2;
- реалізувати генерацію виборцем випадкового ID, поділ ID кандидата на два множники, окреме шифрування обох бюлетенів відкритим ключем ЦВК, формування та підписання двох повідомлень EDS(RSA(EVP), ID);
- реалізувати перевірку ВК наявності виборця у списку та ЕЦП, відхилення некоректних повідомлень, збереження та публікацію прийнятих частин;
- реалізувати зіставлення ЦВК частин за ID, їх об’єднання до розшифрування, розшифрування, перевірку отриманого ID, підрахунок голосів і публікацію результатів;
- перевірити негативні сценарії: незареєстрований виборець, неправильний ЕЦП, зміна частини бюлетеня однією ВК, а також перевірки виборцем публікацій ВК-1, ВК-2 і ЦВК.

2. Короткий опис програмної реалізації

Протокол реалізовано мовою TypeScript і виконується середовищем Node.js без етапу збірки. Криптографічні бібліотеки не використовувалися: модульна арифметика, генерація простих чисел, розширений алгоритм Евкліда, RSA, хеш квадратичної згортки та ЕЦП написані вручну (модулі арифметики та ЕЦП перенесено з лабораторної роботи №1). Довжина модулів RSA — 512 біт, усі великі числа представлені вбудованим типом bigint. Кожна сторона протоколу — окремий модуль, а дані між ними передаються лише у вигляді повідомлень і публікацій, описаних у методичних вказівках: ЦВК передає ВК список виборців, виборець надсилає кожній ВК по одному підписаному повідомленню, ВК публікують пари «ID — зашифрована частина», ЦВК працює тільки з опублікованими даними.

Модуль	Призначення
src/bigint-math.ts	модульне піднесення до степеня, розширений алгоритм Евкліда, обернене за модулем, тест Міллера — Рабіна, генерація простих чисел
src/rsa.ts	генерація ключів, шифрування і розшифрування числа, гомоморфне об’єднання шифротекстів, хеш квадратичної згортки, ЕЦП
src/split.ts	розклади ID кандидата на два множники та відбір розкладів, що не видають вибір
src/types.ts	структури повідомлень і публікацій, канонічна серіалізація EDS(RSA(EVP), ID), причини відхилення
src/voter.ts	виборець: ключі ЕЦП, випадковий ID, поділ і шифрування бюлетеня, підписання; перевірки публікацій ВК і ЦВК
src/central-commission.ts	ЦВК: ID кандидатів, список виборців, ключі шифрування, об’єднання та розшифрування частин, підрахунок
src/part-commission.ts	ВК-1 / ВК-2: прийом списку від ЦВК, перевірки повідомлень, збереження і публікація частин
src/attacks.ts	дії нечесної ВК: підбір множника перебором за відкритим ключем ЦВК
src/main.ts	демонстраційний сценарій: позитивні та негативні випадки, атаки комісій, зведення властивостей
tests/	16 модульних тестів (вбудований тест-ранер node:test)

Таблиця 1 — Структура програмної реалізації

2.1. Мультиплікативна гомоморфність RSA

Частини бюлетеня шифруються «шкільним» RSA без доповнень: c = m^e mod n. Саме така форма зберігає властивість RSA(a) · RSA(b) = (a^e · b^e) mod n = (a · b)^e mod n = RSA(a · b), тому ЦВК може перемножити дві зашифровані частини і отримати шифротекст повного ID кандидата, не розшифровуючи частин окремо. Програма відтворює таблицю з методичних вказівок для ключів (7, 33) і (3, 33) (рисунок 1).

```
/** Шифрування числа відкритим ключем: c = m^e mod n. */
export function encrypt(message: bigint, key: PublicKey): bigint {
  if (message <= 0n || message >= key.n) throw new RangeError("Повідомлення
має бути в діапазоні 0 < m < n");
  return modPow(message, key.e, key.n);
}

/** Розшифрування закритим ключем: m = c^d mod n. */
export function decrypt(cipher: bigint, key: PrivateKey): bigint {
  return modPow(cipher, key.d, key.n);
}

/**
 * Об'єднання шифротекстів без розшифрування:
 * (m₁^e · m₂^e) mod n = (m₁ · m₂)^e mod n, тобто RSA(m₁) · RSA(m₂) =
RSA(m₁ · m₂).
 */
export function multiplyCiphers(first: bigint, second: bigint, key:
PublicKey): bigint {
  return (first * second) % key.n;
}
```

Лістинг 1 — Шифрування, розшифрування та об’єднання шифротекстів (src/rsa.ts)

2.2. ID кандидатів і поділ на множники

ЦВК надає кандидатам ID вигляду 12 · k, тобто 24 і 36. Їх спільні дільники 2, 3, 4, 6 і 12 трапляються в розкладах обох кандидатів, тому окремий множник не дозволяє однозначно визначити вибір. Множник вважається таким, що приховує вибір, якщо він ділить ID щонайменше двох кандидатів. Виборець випадково обирає один із розкладів, у яких обидва множники приховують вибір: для 24 це 2 × 12, 4 × 6, 6 × 4, 12 × 2, для 36 — 3 × 12, 6 × 6, 12 × 3. Розклади на кшталт 3 × 8 чи 4 × 9 відкидаються, бо множник 8 трапляється лише в 24, а 9 — лише в 36.

```
/** Усі впорядковані розклади id = a · b з нетривіальними множниками (a, b
> 1). */
export function factorPairs(id: bigint): Split[] {
  const pairs: Split[] = [];
  for (let a = 2n; a * 2n <= id; a += 1n) {
    if (id % a === 0n && id / a > 1n) pairs.push([a, id / a]);
  }
  return pairs;
}

/** Кількість ID кандидатів, частиною яких може бути множник. */
export function candidatesSharing(factor: bigint, ids: readonly bigint[]):
number {
  return ids.filter((id) => id % factor === 0n).length;
}

/** Розклади, у яких обидва множники не видають кандидата. */
export function hidingSplits(id: bigint, ids: readonly bigint[]): Split[] {
  return factorPairs(id).filter((pair) => pair.every((factor) =>
candidatesSharing(factor, ids) >= 2));
}
```

Лістинг 2 — Розклади ID та відбір розкладів, що не видають вибір (src/split.ts)

```
/** ЦВК видає кандидатам ID, придатні для поділу: кожен має розклад, що не
видає вибір. */
export function assignCandidateIds(names: readonly string[]): Candidate[] {
  const ids = names.map((_, index) => ID_BASE * (FIRST_MULTIPLIER +
BigInt(index)));
  for (const id of ids) {
    if (hidingSplits(id, ids).length === 0) throw new Error(`ID ${id} не
можна поділити, не видавши вибір`);
  }
  return names.map((name, index) => ({ id: ids[index]!, name }));
}
```

Лістинг 3 — Надання кандидатам ID (src/central-commission.ts)

2.3. Дії виборця

Виборець самостійно генерує пару ключів RSA для ЕЦП і надсилає відкритий ключ до ЦВК. Під час голосування він генерує випадковий 64-бітний ID (спільний для обох повідомлень), ділить ID обраного кандидата на два множники, окремо шифрує кожен відкритим ключем ЦВК, додає до кожного шифротексту свій ID і підписує обидва повідомлення власним закритим ключем. Перше повідомлення надсилається до ВК-1, друге — до ВК-2, разом із ПІБ. Собі виборець залишає квитанцію: свій ID, вибір, розклад і обидва шифротексти.

```
/** Кроки 2–3: поділ ID кандидата, шифрування множників, EDS(RSA(EVP), ID)
для кожної ВК. */
vote(
  candidateId: bigint,
  candidateIds: readonly bigint[],
  cvkKey: PublicKey,
  split: Split = pickSplit(candidateId, candidateIds),
): Ballot {
  if (split[0] * split[1] !== candidateId) throw new Error(`${split.join("
· ")} ≠ ${candidateId}`);

  const voterId = randomBigInt(VOTER_ID_MIN, VOTER_ID_MAX);
  const encryptedParts = [encrypt(split[0], cvkKey), encrypt(split[1],
cvkKey)] as const;
  const messages = [
    this.signPart({ voterId, encryptedPart: encryptedParts[0] }),
    this.signPart({ voterId, encryptedPart: encryptedParts[1] }),
  ] as const;

  return { messages, receipt: { voterId, candidateId, split, encryptedParts
} };
}

signPart(part: BallotPart): PartMessage {
  return { senderName: this.name, part: { ...part, signature:
sign(serializePart(part), this.keys.privateKey) } };
}
```

Лістинг 4 — Формування двох підписаних повідомлень EDS(RSA(EVP), ID) (src/voter.ts)

```
/** Канонічний текст частини бюлетеня — саме він хешується та підписується.
*/
export function serializePart(part: BallotPart): string {
  return JSON.stringify({ voterId: part.voterId.toString(), encryptedPart:
part.encryptedPart.toString(16) });
}
```

Лістинг 5 — Канонічний текст повідомлення, що підписується (src/types.ts)

2.4. Перевірки ВК-1 та ВК-2

Кожна ВК отримує від ЦВК список виборців з відкритими ключами ЕЦП. Для кожного повідомлення вона перевіряє наявність відправника у списку, справжність ЕЦП (хеш квадратичної згортки H порівнюється з H_c = ЕЦП^e mod n) та відсутність уже прийнятої частини від цього виборця. Прийнята частина зберігається разом з ID виборця, а після завершення голосування ВК публікує всі пари «ID — зашифрована частина» без ПІБ. Закритого ключа ЦВК у ВК немає, тому розшифрувати свою частину вона не може.

```
/** Крок 4: перевірка виборця за списком, перевірка ЕЦП, відмітка про
отримання. */
receive(message: PartMessage): VkResult {
  const { senderName, part } = message;

  const voter = this.voters.get(senderName);
  this.note("inList", voter !== undefined, voter ? `«${senderName}» є у
списку від ЦВК` : `«${senderName}» відсутній у списку`);
  if (!voter) return this.reject(senderName, "NOT_IN_LIST");

  const signatureValid = verify(serializePart(part), part.signature,
voter.publicKey);
  this.note("signature", signatureValid, signatureValid ? "H = H_c, ЕЦП
справжній" : "H ≠ H_c, ЕЦП недійсний");
  if (!signatureValid) return this.reject(senderName, "BAD_SIGNATURE");

  const alreadyVoted = this.received.has(senderName);
  this.note(
    "notVotedYet",
    !alreadyVoted,
    alreadyVoted ? `від «${senderName}» частину вже прийнято` : "відмітку
про отримання поставлено",
  );
  if (alreadyVoted) return this.reject(senderName, "ALREADY_VOTED");

  this.received.set(senderName, { voterId: part.voterId, encryptedPart:
part.encryptedPart });
  return { status: "accepted", voterName: senderName, voterId: part.voterId
};
}

/** Крок 5: публікація всіх ID і відповідних зашифрованих частин (без ПІБ).
*/
publish(): BallotPart[] {
  return [...this.received.values()];
}
```

Лістинг 6 — Обробка та публікація частин бюлетеня (src/part-commission.ts)

2.5. Об’єднання, розшифрування та підрахунок у ЦВК

ЦВК працює лише з опублікованими даними ВК-1 і ВК-2. Частини одного виборця зіставляються за ID; якщо для ID опубліковано лише одну частину, бюлетень відкидається. Далі ЦВК спочатку об’єднує шифротексти $C = C_1 \cdot C_2 \bmod n$ і лише потім розшифровує результат своїм закритим ключем: $C^d \bmod n = m_1 \cdot m_2$. Якщо отримане число не є ID кандидата, бюлетень відкидається. Після підрахунку ЦВК публікує результати разом з ID виборців і відновленими бюлетенями.

```
/**
 * Крок 6: зіставлення частин за ID виборця, об'єднання C₁ · C₂ mod n
 * і лише потім розшифрування: (C₁ · C₂)^d mod n = m₁ · m₂.
 */
combine(first: readonly BallotPart[], second: readonly BallotPart[]):
CombinedBallot[] {
  const index = (list: readonly BallotPart[]) => new Map(list.map((entry)
=> [entry.voterId, entry.encryptedPart]));
  const [firstById, secondById] = [index(first), index(second)];
  const voterIds = [...new Set([...firstById.keys(),
...secondById.keys()])];

  return voterIds.map((voterId): CombinedBallot => {
    const parts = [firstById.get(voterId), secondById.get(voterId)] as
const;
    const [c1, c2] = parts;
    if (c1 === undefined || c2 === undefined) return { voterId, parts,
status: "rejected", reason: "MISSING_PART" };

    const combined = multiplyCiphers(c1, c2, this.publicKey);
    const ballot = decrypt(combined, this.keys.privateKey);
    return this.candidates.some((candidate) => candidate.id === ballot)
      ? { voterId, parts, status: "accepted", combined, ballot }
      : { voterId, parts, status: "rejected", reason: "NOT_A_CANDIDATE",
combined, ballot };
  });
}

/** Підрахунок прийнятих бюлетенів; разом з результатами публікуються ID
виборців та їх бюлетені. */
tally(ballots: readonly CombinedBallot[]): ElectionResults {
  const tally = new Map(this.candidates.map((candidate) => [candidate.id,
0]));
  for (const ballot of ballots) {
    if (ballot.status === "accepted") tally.set(ballot.ballot,
(tally.get(ballot.ballot) ?? 0) + 1);
  }

  const max = Math.max(0, ...tally.values());
  const leaders = max === 0 ? [] : this.candidates.filter((candidate) =>
tally.get(candidate.id) === max);
  return { ballots, tally, leaders, isTie: leaders.length > 1 };
}
```

Лістинг 7 — Об’єднання частин і підрахунок голосів (src/central-commission.ts)

2.6. Перевірки виборця та дії нечесної ВК

За квитанцією виборець перевіряє, що ВК-1 і ВК-2 опублікували для його ID саме ті шифротексти, які він надіслав, і що ЦВК опублікувала для його ID саме того кандидата, за якого він голосував. Для моделювання атак додано функцію підбору множника: оскільки RSA без доповнень детермінований, а відкритий ключ ЦВК доступний усім, ВК може зашифрувати числа 1…36 і порівняти результати зі своїм шифротекстом.

```
/** Перевірка виборцем, що кожна ВК опублікувала саме ту зашифровану
частину, яку він їй надіслав. */
export function checkPublications(
  receipt: Receipt,
  published: readonly [readonly BallotPart[], readonly BallotPart[]],
): readonly [boolean, boolean] {
  const matches = (list: readonly BallotPart[], expected: bigint) =>
    list.some((entry) => entry.voterId === receipt.voterId &&
entry.encryptedPart === expected);
  return [matches(published[0], receipt.encryptedParts[0]),
matches(published[1], receipt.encryptedParts[1])];
}

/** Перевірка виборцем, що ЦВК опублікувала для його ID саме того
кандидата, за якого він голосував. */
export function checkResult(receipt: Receipt, ballots: readonly
CombinedBallot[]): boolean {
  const own = ballots.find((ballot) => ballot.voterId === receipt.voterId);
  return own?.status === "accepted" && own.ballot === receipt.candidateId;
}

/**
 * Дії нечесної ВК. RSA без доповнень детермінований, а відкритий ключ ЦВК
 * доступний усім, тому малий множник можна «розшифрувати» перебором.
 */
export function decodeByDictionary(cipher: bigint, key: PublicKey,
maxValue: bigint): bigint | undefined {
  for (let value = 1n; value <= maxValue; value += 1n) {
    if (encrypt(value, key) === cipher) return value;
  }
  return undefined;
}
```

Лістинг 8 — Перевірки виборця (src/voter.ts) і підбір множника (src/attacks.ts)

3. Результати виконання протоколу та передбачених перевірок

Демонстраційний сценарій запускається командою npm start і складається з трьох частин: перевірки гомоморфності RSA на прикладі з методичних вказівок, штатного голосування п’яти виборців з негативними сценаріями перевірок ВК, а також окремої сесії, у якій нечесні комісії змінюють опубліковані дані. Зведення сценаріїв наведено в таблиці 2.

Сценарій	**Що перевіряється**	**Результат**
Коректне голосування 5 виборців	усі перевірки ВК, об’єднання і розшифрування	5 бюлетенів прийнято, 24 : 36 = 3 : 2
Незареєстрований виборець	наявність у списку	ВК-1 і ВК-2: NOT_IN_LIST
Підміна RSA(EVP1) по дорозі	ЕЦП повідомлення	ВК-1: BAD_SIGNATURE
Повідомлення з чужим ЕЦП	ЕЦП ключем виборця зі списку	ВК-2: BAD_SIGNATURE
Повторне голосування	відмітка про отримання	ВК-1 і ВК-2: ALREADY_VOTED
Перевірка публікацій ВК	квитанція виборця	усі 5 виборців: збігається
Перевірка публікації ЦВК	квитанція виборця	усі 5 виборців: збігається
ВК-1 підміняє 12 → 18	об’єднання і перевірка ID	24 → 36 зараховано; 36 → 54 відкинуто (NOT_A_CANDIDATE); виборці бачать невідповідність ВК-1
ВК-2 не публікує частину	зіставлення за ID	MISSING_PART; виборець бачить відсутність у ВК-2
Змова ВК-1 і ВК-2	об’єднання і перевірка ID	24 → 36 зараховано; виявляє лише виборець
ЦВК публікує 36 замість 24	квитанція виборця	виборець бачить невідповідність ЦВК

Таблиця 2 — Перевірені сценарії роботи протоколу

На рисунку 1 наведено перевірку мультиплікативної гомоморфності для ключів з методичних вказівок: для всіх чотирьох поділів числа 24 добуток шифротекстів дорівнює RSA(24) = 18, а його розшифрування дає 24.

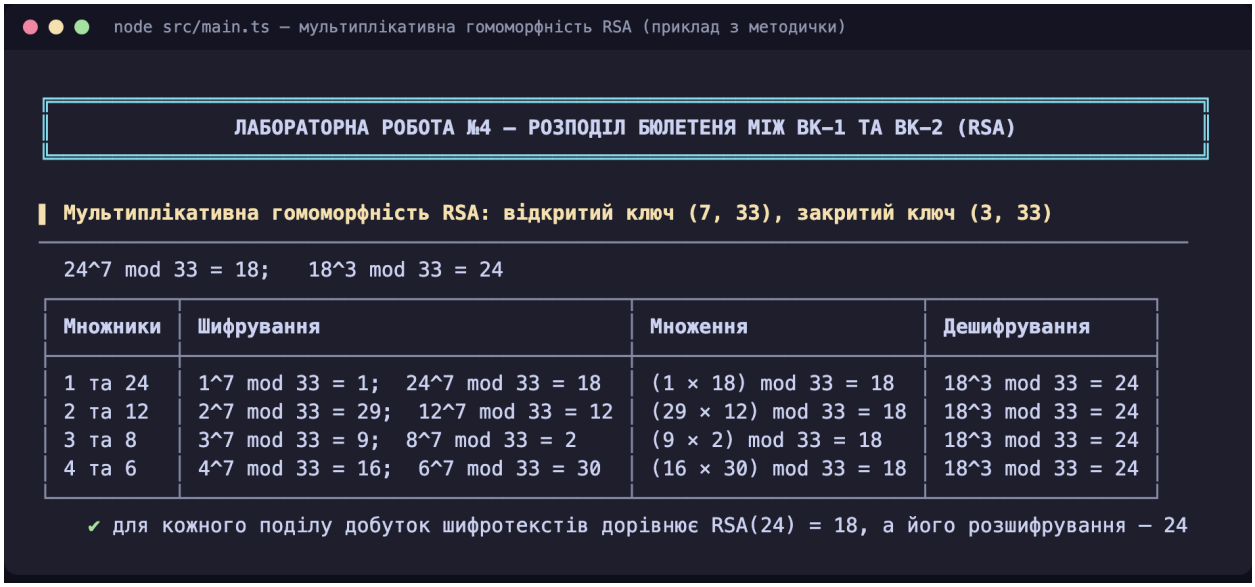


Рисунок 1 — Мультиплікативна гомоморфність RSA (ключі (7, 33) і (3, 33))

На рисунку 2 показано перший крок протоколу: виборці згенерували ключі ЕЦП і надіслали відкриті ключі до ЦВК, ЦВК надала кандидатам ID 24 і 36 та опублікувала їх разом з розкладами, що не видають вибір, згенерувала ключі шифрування, опублікувала відкритий ключ і передала список виборців до ВК-1 та ВК-2.

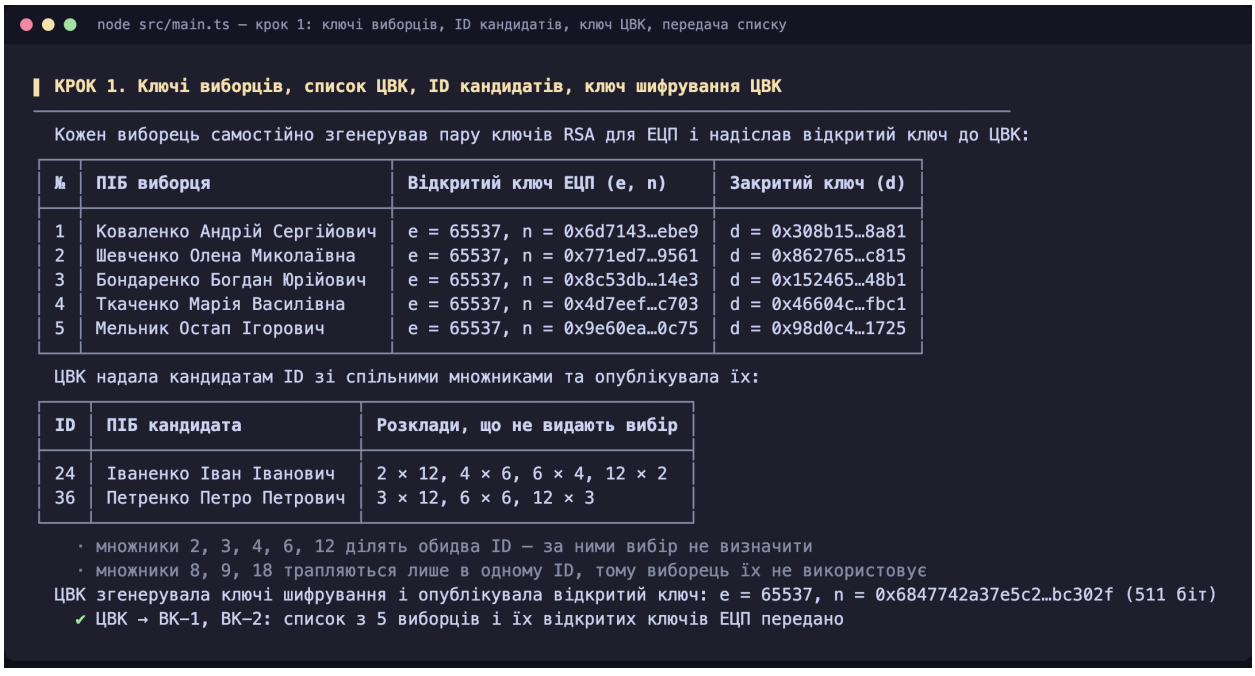


Рисунок 2 — Крок 1: ключі виборців, ID кандидатів, ключ ЦВК, передача списку до ВК

Рисунок 3 демонструє позитивний сценарій для двох виборців: випадковий ID виборця, поділ ID кандидата, шифрування кожного множника

відкритим ключем ЦВК, ЕЦП обох повідомлень і проходження всіх перевірок у ВК-1 та ВК-2.

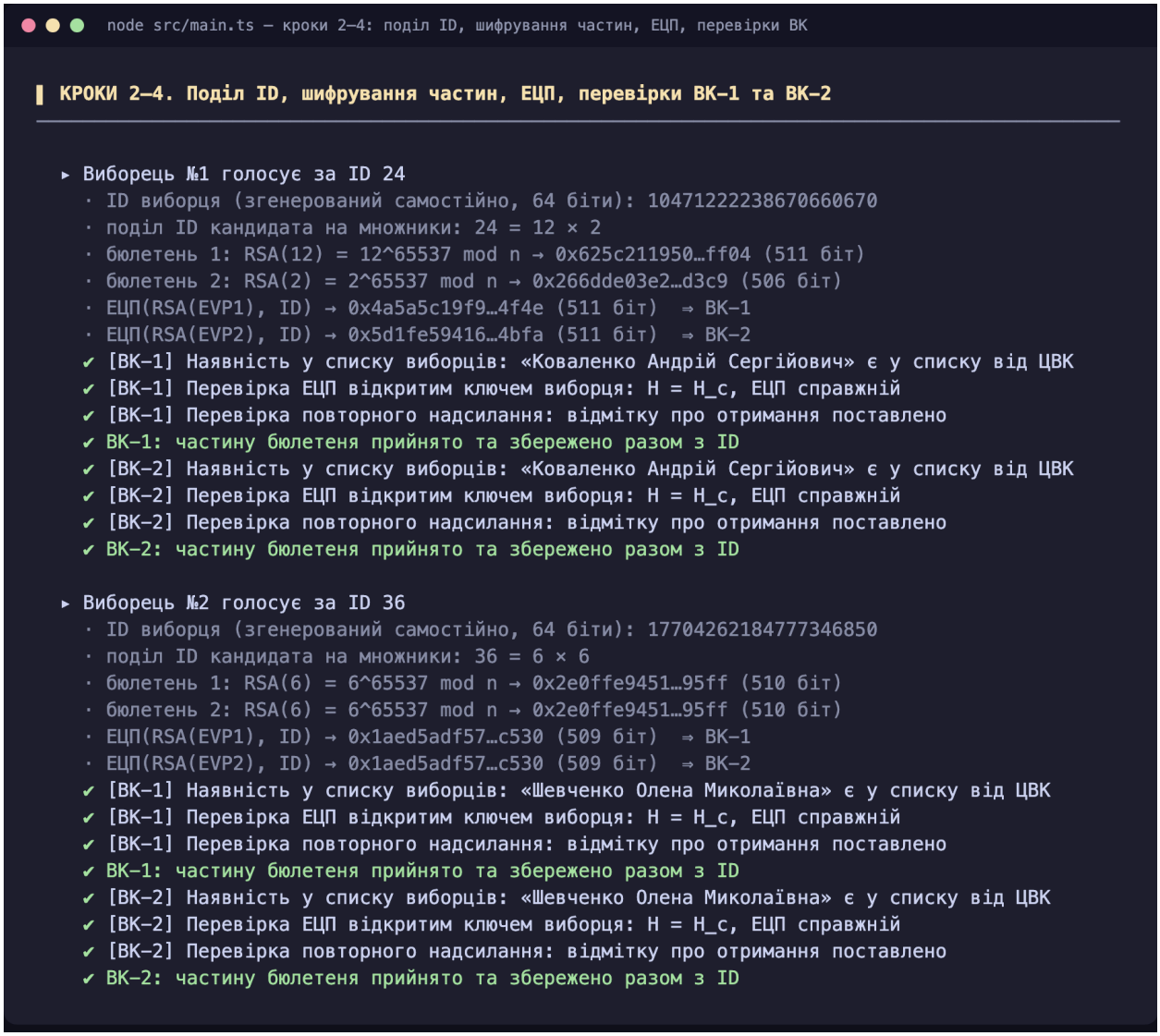


Рисунок 3 — Кроки 2–4: поділ ID, шифрування частин, ЕЦП і перевірки ВК

Рисунки 4 і 5 демонструють негативні сценарії перевірок ВК. Повідомлення від особи, якої немає у списку ЦВК, відхиляються обома комісіями. Підмінену по дорозі частину бюлетеня технічно можна зашифрувати відкритим ключем ЦВК, але ЕЦП виборця вже не відповідає тексту, тому ВК-1 відхиляє повідомлення; оригінальне повідомлення того самого виборця приймається. Повідомлення, підписане ключем іншого виборця, та повторне голосування також відхиляються.

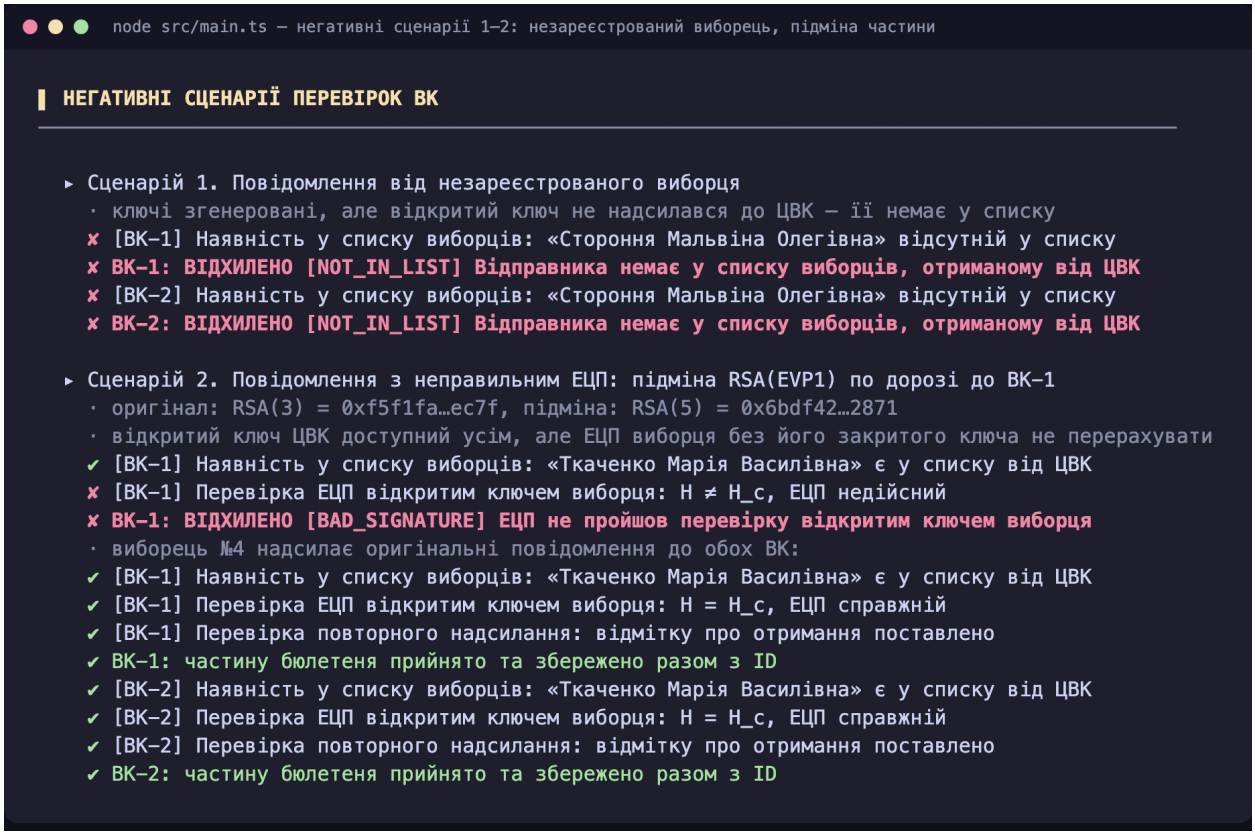


Рисунок 4 — Негативні сценарії: незареєстрований виборець, підміна частини бюлетеня

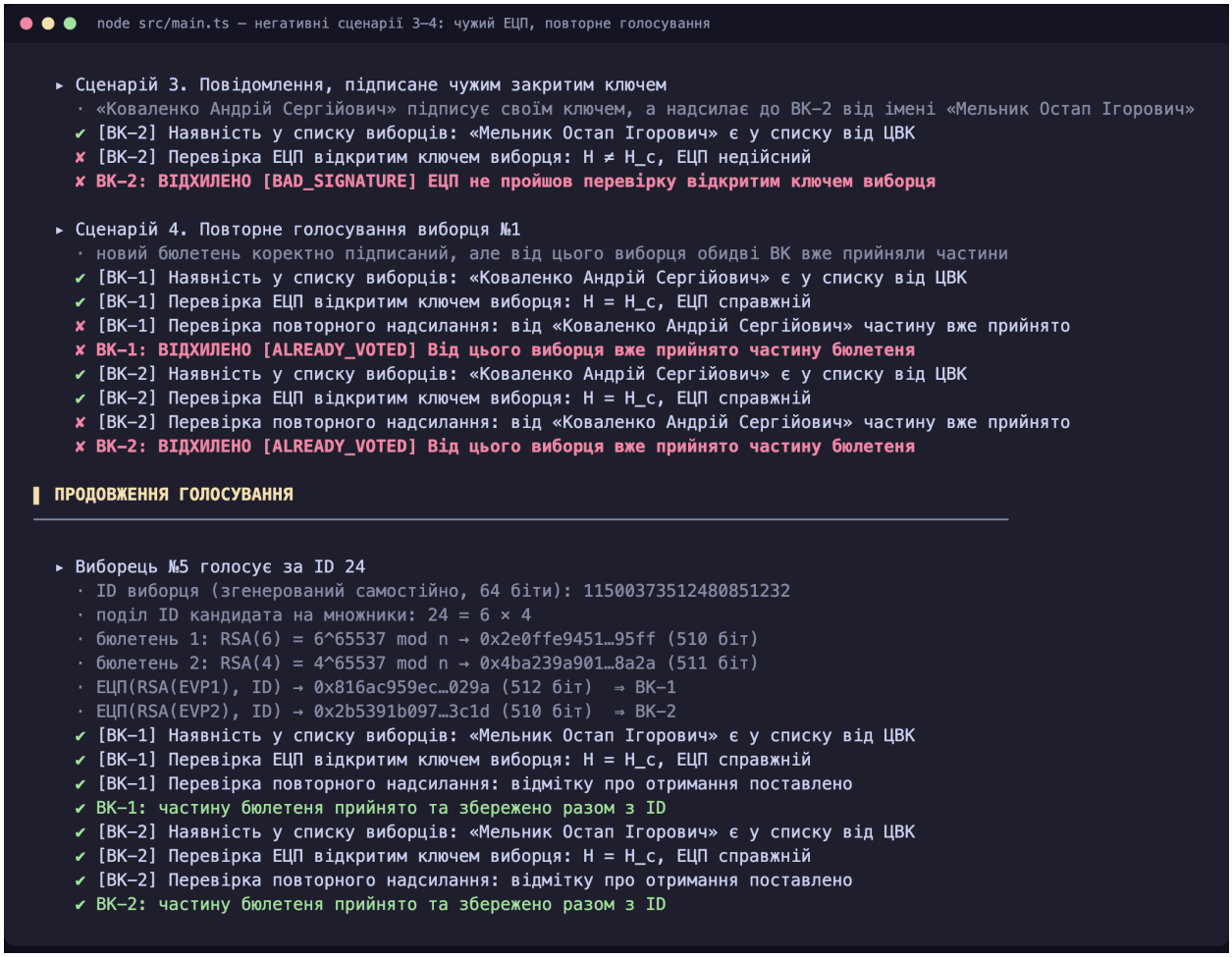


Рисунок 5 — Негативні сценарії: чужий ЕЦП, повторне голосування; голосування виборця №5

На рисунку 6 ВК-1 та ВК-2 публікують ID виборців і зашифровані частини без ПІБ, а кожен виборець переконується, що опубліковано саме те, що він надсилав. Варто звернути увагу, що однакові множники дають однакові шифротексти: RSA без доповнень детермінований, і це використовується в атаці з підбором множника.

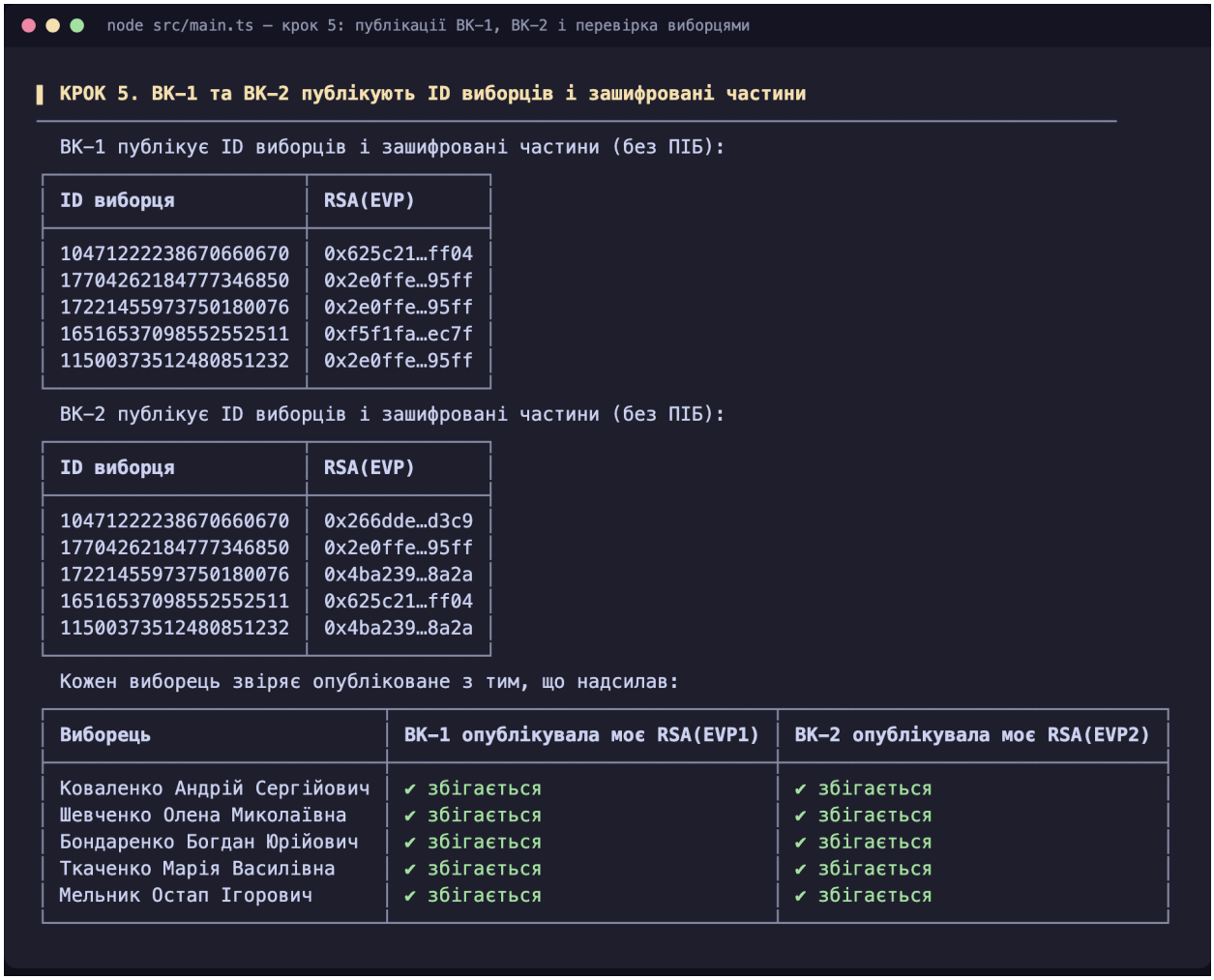


Рисунок 6 — Крок 5: публікації ВК-1, ВК-2 та їх перевірка виборцями

На рисунку 7 ЦВК зіставляє частини за ID, об’єднує їх, розшифровує вже об’єднаний бюлетень і отримує початкові ID кандидатів. Підсумок голосування 3 : 2 на користь кандидата з ID 24; кожен виборець перевіряє, що опублікований ЦВК бюлетень збігається з його вибором.

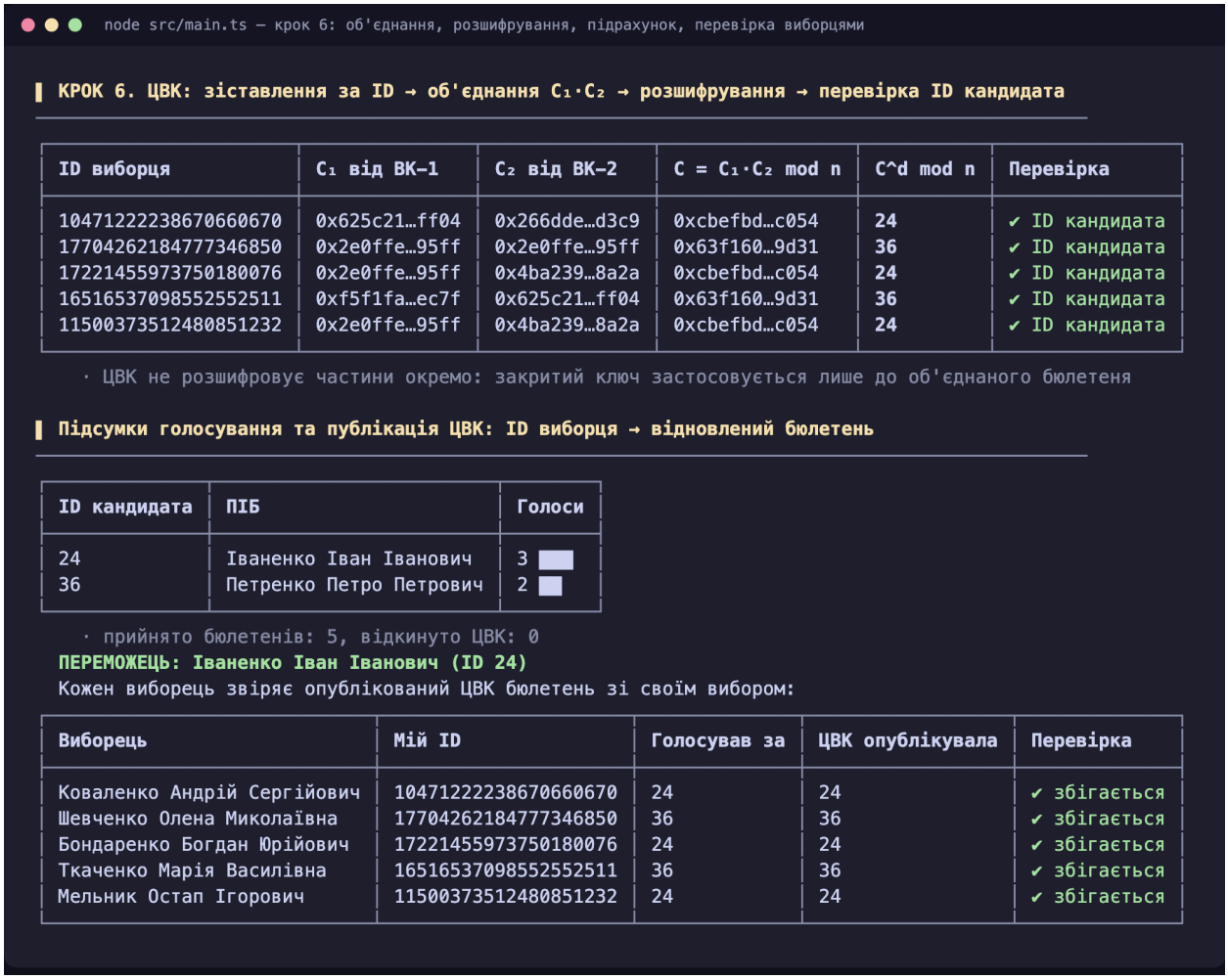


Рисунок 7 — Крок 6: об’єднання та розшифрування, підрахунок, перевірка виборцями

Рисунки 8–10 відносяться до окремої сесії з атаками, у якій виборці використовують фіксовані розклади. ВК-1 підбирає всі свої множники перебором, але кожен з них ділить обидва ID, тому вибір виборця не визначається. Намагаючись перетворити голоси за 24 на голоси за 36, ВК-1 замінює множник 12 на 18: для виборця з розкладом 12 × 2 підміна дає коректний ID 36, а для виборця з розкладом 12 × 3 — число 54, яке не є ID кандидата і відкидається ЦВК. ВК-2 не публікує одну частину — такий бюлетень відкидається через відсутність пари.

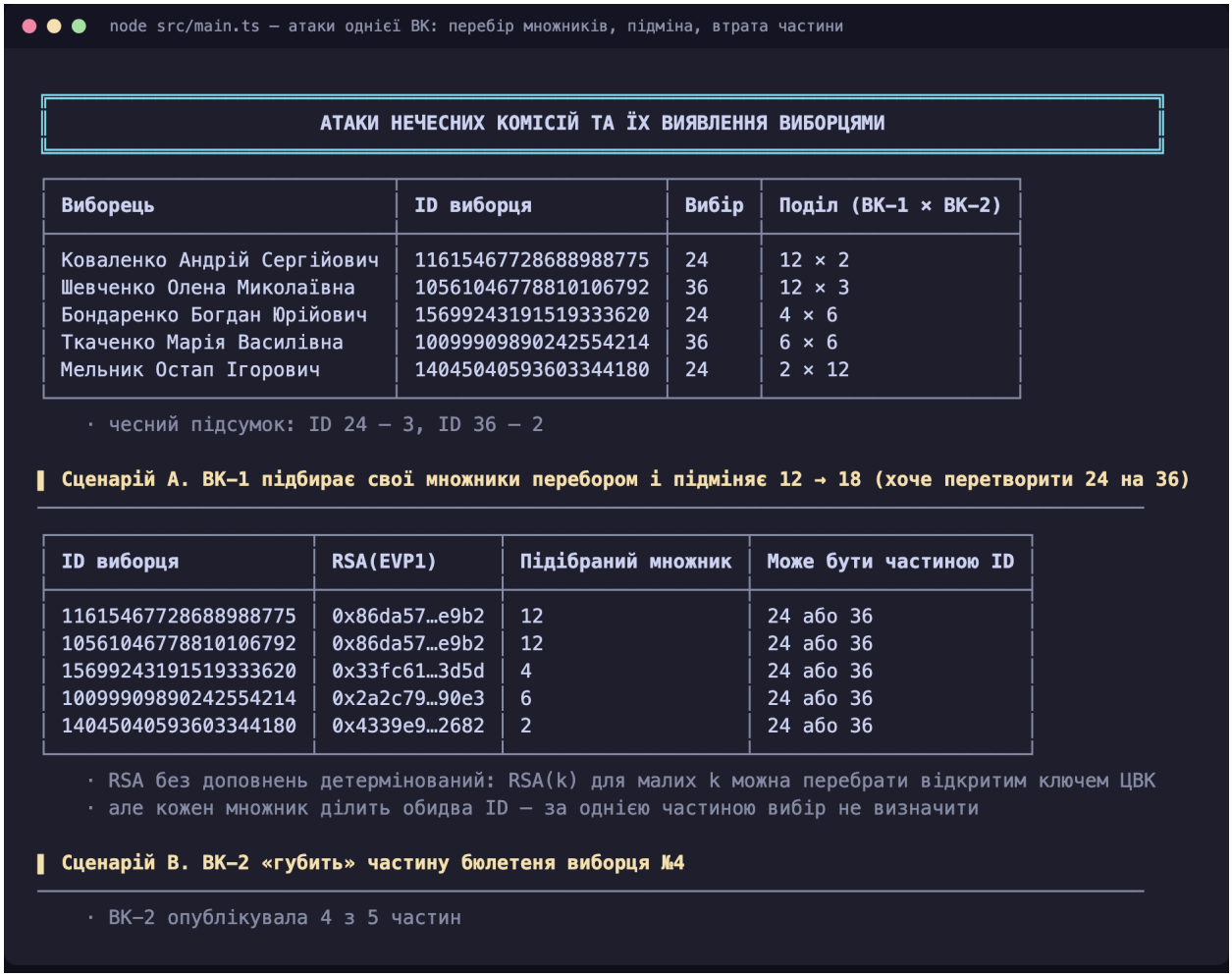


Рисунок 8 — Атаки однієї ВК: підбір множників, підміна 12 → 18, втрата частини

При змові ВК-1 і ВК-2 обидві частини виборця замінено на RSA(6), і після об’єднання ЦВК отримує коректний ID 36 — перевірка ЦВК цього не виявляє. Додатково змодельовано нечесну ЦВК, яка публікує 36 замість 24. У сумі атаки змінюють переможця: замість 3 : 2 на користь 24 опубліковано 0 : 3.

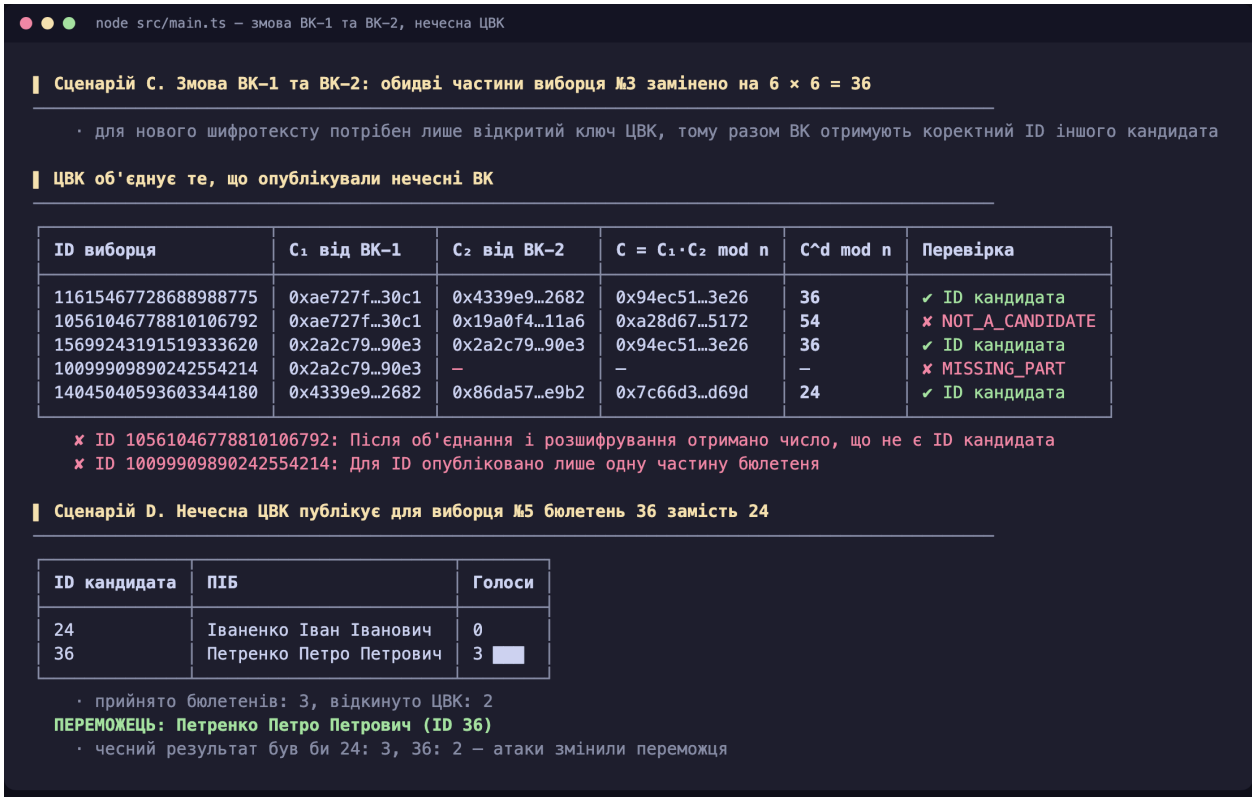


Рисунок 9 — Змова ВК-1 та ВК-2, нечесна ЦВК, результати після атак

На рисунку 10 показано виявлення атак: виборці №1–3 бачать, що ВК-1 опублікувала не їх шифротекст, виборець №4 — що ВК-2 не опублікувала його частину, виборець №5 — що ЦВК опублікувала інший бюлетень. Нижче наведено зведення властивостей протоколу.



Рисунок 10 — Виявлення атак виборцями та властивості протоколу

4. Тестування

Реалізацію покрито 16 модульними тестами (команда npm test). Тести відтворюють таблицю гомоморфності з методичних вказівок, перевіряють об’єднання шифротекстів для згенерованого ключа, ЕЦП, підбір множника, ID кандидатів і відбір розкладів, перевірки ВК (незареєстрований виборець, підміна частини, чужий ЕЦП, повторне голосування), об’єднання та підрахунок у ЦВК, відкидання бюлетенів NOT_A_CANDIDATE і MISSING_PART, а також перевірки виборцем публікацій ВК і ЦВК.

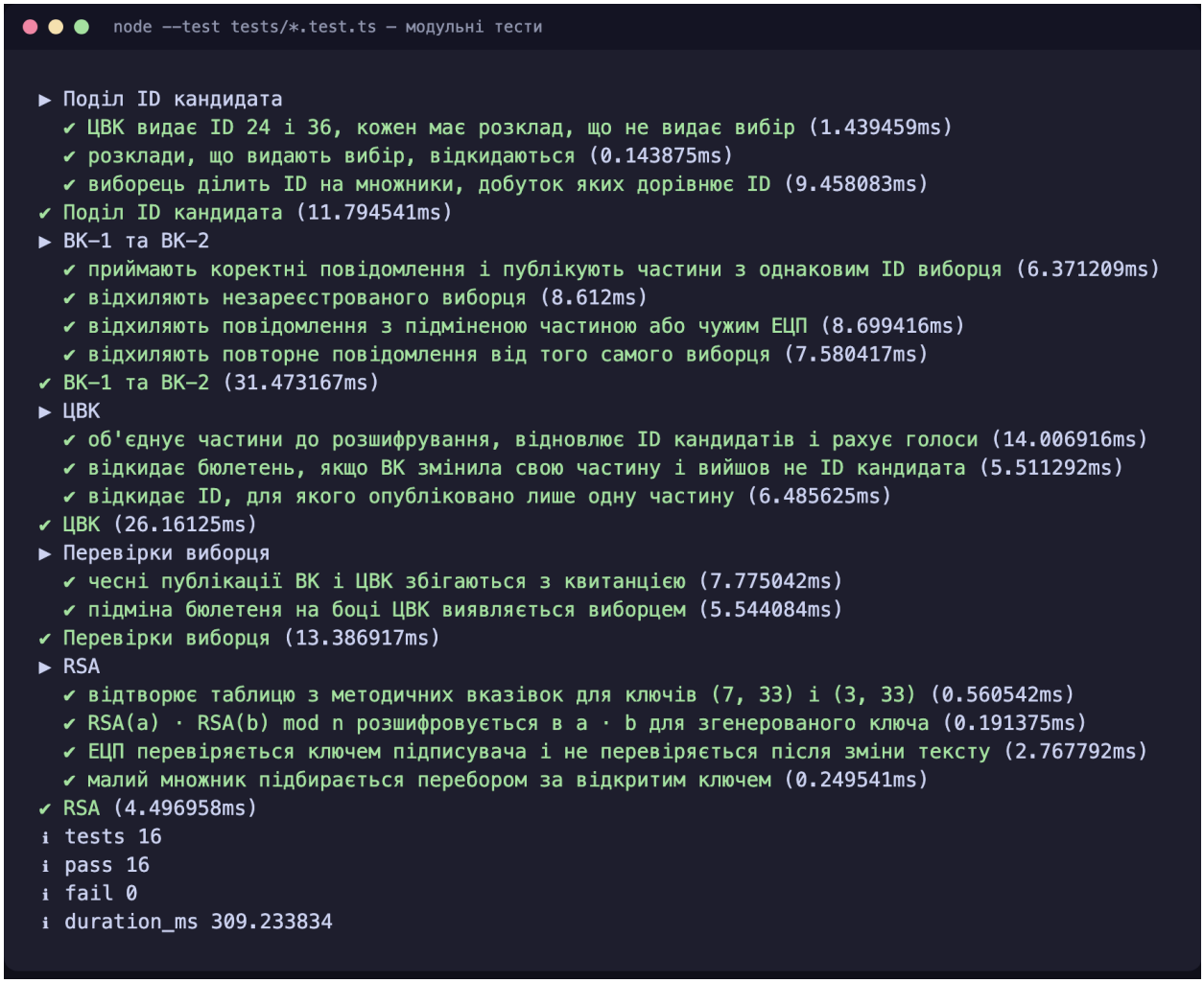


Рисунок 11 — Результати запуску модульних тестів

5. Висновок

У ході виконання лабораторної роботи програмно реалізовано протокол електронного голосування з розділенням бюлетеня між ВК-1 та ВК-2 для двох кандидатів і п’яти виборців. ЦВК, ВК-1 та ВК-2 реалізовані окремими модулями, які обмінюються лише передбаченими протоколом повідомленнями та публікаціями. Для шифрування частин бюлетеня і ЕЦП використано RSA; мультиплікативна гомоморфність RSA дозволила ЦВК об’єднувати зашифровані частини до розшифрування і відновлювати початковий ID кандидата.

Експериментально підтверджено, що ВК відхиляють повідомлення незареєстрованих виборців, повідомлення з неправильним ЕЦП і повторні повідомлення, а ЦВК відкидає бюлетені з відсутньою частиною або з числом, що не є ID кандидата. Підміна частини однією ВК без знання другої частини дає непередбачуваний результат: частина бюлетенів стає некоректною і

сигналізує про втручання, а решта підмін виявляється виборцями за квитанціями.

Водночас протокол має суттєві обмеження. Змова ВК-1 і ВК-2 дозволяє узгоджено замінити обидві частини на коректний ID іншого кандидата, і виявити це може лише сам виборець. Оскільки RSA без доповнень детермінований, а відкритий ключ ЦВК загальнодоступний, малий множник підбирається перебором; таємницю вибору зберігає лише те, що кожен множник ділить ID обох кандидатів. Із двома кандидатами і цілими множниками повна невизначеність недосяжна, тому для практичних систем доцільніше використовувати адитивно гомоморфне шифрування з пороговим розшифруванням (наприклад, Paillier) або імовірнісне шифрування.

Відхилення від методичних вказівок: окрім обов'язкових сценаріїв додатково змодельовано атаку з підбором множника, втрату частини бюлетеня однією ВК і нечесну ЦВК; виборець обирає лише ті розклади ID, у яких обидва множники трапляються в ID обох кандидатів.

6. Відповіді на контрольні запитання

1. Опишіть послідовність роботи протоколу електронного голосування з розподілом бюлетеня між ВК-1 та ВК-2. Виборці генерують ключі ЕЦП і надсилають відкриті ключі до ЦВК; ЦВК формує списки виборців і кандидатів, надає кандидатам ID зі спільними множниками, публікує їх, генерує ключі шифрування, публікує відкритий ключ і передає список виборців до ВК-1 та ВК-2. Виборець генерує випадковий ID, ділить ID обраного кандидата на два множники, шифрує кожен відкритим ключем ЦВК, додає свій ID, підписує обидва повідомлення і надсилає одне до ВК-1, друге — до ВК-2. ВК перевіряють виборця за списком та ЕЦП, зберігають частини з ID і після голосування публікують їх. ЦВК зіставляє частини за ID, перемножує шифротексти, розшифровує результат, перевіряє, що отримано ID кандидата, рахує голоси і публікує результати разом з ID виборців та їх бюлетенями.

2. Для чого ID кандидата розділяється на множники та які вимоги повинні виконуватися до вибраних ID кандидатів? Поділ потрібен, щоб жодна окрема ВК не мала повного бюлетеня: вона контролює лише одну частину і не може ні дізнатися вибір, ні цілеспрямовано його змінити. ID кандидатів

мають бути складеними числами з багатьма спільними множниками, щоб кожен множник траплявся в розкладах щонайменше двох кандидатів (у роботі — 24 і 36 зі спільними множниками 2, 3, 4, 6, 12); добуток множників має бути меншим за модуль n.

3. Чому окрема частина бюлетеня не повинна дозволяти однозначно визначити обраного кандидата? Інакше порушується таємниця голосування: ВК бачить ПІБ відправника, тож знаючи множник, вона дізналася б вибір конкретної людини. Крім того, знаючи вибір, ВК могла б цілеспрямовано підмінити свою частину. Наприклад, при розкладі 24 = 1 × 24 комісія, що отримала 24, одразу знає вибір; тому виборець використовує лише множники, спільні для різних ID.

4. У чому полягає властивість мультиплікативної гомоморфності RSA та як вона використовується для об'єднання частин бюлетеня? Добуток шифротекстів є шифротекстом добутку: RSA(a) · RSA(b) = a^e · b^e = (a · b)^e mod n. ЦВК перемножує зашифровані частини $C_1 \cdot C_2 \bmod n$ і отримує RSA(ID кандидата), який потім розшифровує: $(C_1 \cdot C_2)^d \bmod n = a \cdot b$. Наприклад, для ключів (7, 33) і (3, 33): RSA(4) · RSA(6) = 16 · 30 mod 33 = 18 = RSA(24).

5. Чому в даному протоколі спочатку виконується об'єднання зашифрованих частин бюлетеня, а вже після цього їх розшифрування? Щоб окремі множники не розкривалися нікому, зокрема ЦВК: вона бачить лише кінцевий ID кандидата, прив'язаний до анонімного ID виборця, а не те, як виборець поділив свій вибір. Окреме розшифрування не дало б жодної переваги, але розкрило б проміжні значення і дозволило б зіставляти розклади. Гомоморфність робить об'єднання без розшифрування можливим.

6. Як виборець може перевірити правильність публікації своїх частин бюлетеня ВК-1 та ВК-2 і правильність відновлення бюлетеня ЦВК? Виборець зберігає свій ID, обидва шифротексти і свій вибір. У публікаціях ВК-1 та ВК-2 він знаходить свій ID і порівнює опубліковані шифротексти з надісланими. У публікації ЦВК він знаходить свій ID і перевіряє, що відновлений бюлетень дорівнює ID обраного кандидата. Будь-яка розбіжність — доказ для скарги.

7. Які можливості для шахрайства має окрема ВК та чому підміна лише однієї частини бюлетеня може бути виявлена? Окрема ВК може не

опублікувати частину, підмінити її новим шифротекстом (відкритий ключ ЦВК доступний усім) або підібрати свій малий множник перебором. Оскільки другу частину вона не знає, результат підміни непередбачуваний: у демонстрації заміна 12 на 18 перетворила 24 на 36, але 36 на 54 — число, що не є ID кандидата, і ЦВК його відкинула. Велика кількість таких бюлетенів сигналізує про втручання, а кожну підміну додатково виявляє виборець, порівнюючи публікацію ВК зі своєю квитанцією.

8. Що змінюється у випадку змови ВК-1 та ВК-2 і які обмеження цього протоколу залишаються? Разом комісії можуть узгоджено замінити обидві частини (наприклад, на RSA(6) і RSA(6)), отримавши після об’єднання коректний ID іншого кандидата, тож перевірка ЦВК підміни не виявить; лишається тільки перевірка виборцем, яка спрацьовує лише якщо він її виконає і поскаржиться. Також залишаються: детермінованість RSA і можливість перебору множників, неповна таємниця при двох кандидатах, довіра до ЦВК, яка тримає закритий ключ і може пов’язати дані у змові з ВК, та неможливість для виборця довести, що квитанція справжня, без додаткових механізмів.